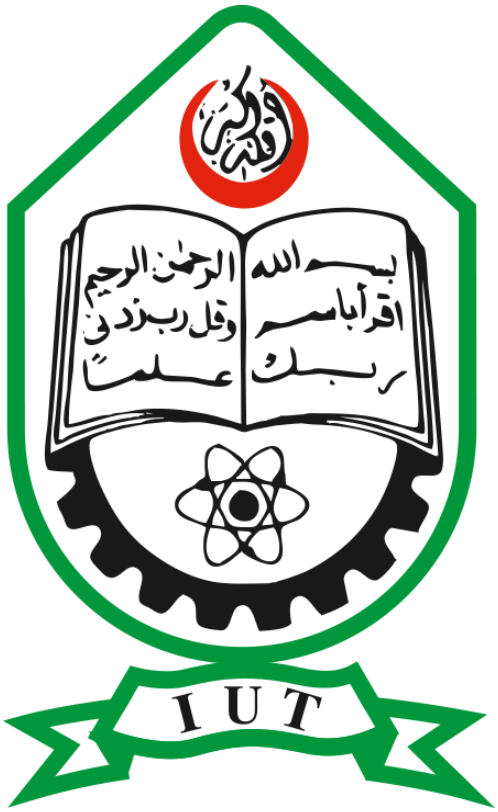
Islamic University of Technology

Organization of Islamic Cooperation



Name	**:** Antara Arifa Mullick
ID	**:** 210042132
Program	**:** SWE
Department	**:** CSE

Subject	**:** Artificial Intelligence
Subject Code	**:** CSE 4618
Title	**:** Constraint Satisfaction Problem
Date of Submission	**:** 28th June, 2025

1. Introduction

This lab focused on exploring various Constraint Satisfaction Problems (CSPs) using the AIspace constraint solver tool. The goal was to define variables, domains, and constraints effectively and apply consistency and backtracking algorithms to find valid solutions. Through five diverse problem scenarios—ranging from map coloring to complex scheduling—I gained hands-on experience in formalizing and solving CSPs, learning how constraint-based reasoning supports decision-making in AI.

2. Problem Analysis

2.1 Core Challenges

Each problem presented distinct modeling challenges:

1. **Variable and Domain Design**: Selecting meaningful representations for regions, people, tasks, etc.

2. **Constraint Encoding**: Expressing logical rules via equality, inequality, adjacency, or custom logic.

3. **Search and Inference**: Employing arc consistency and backtracking to reduce the search space.

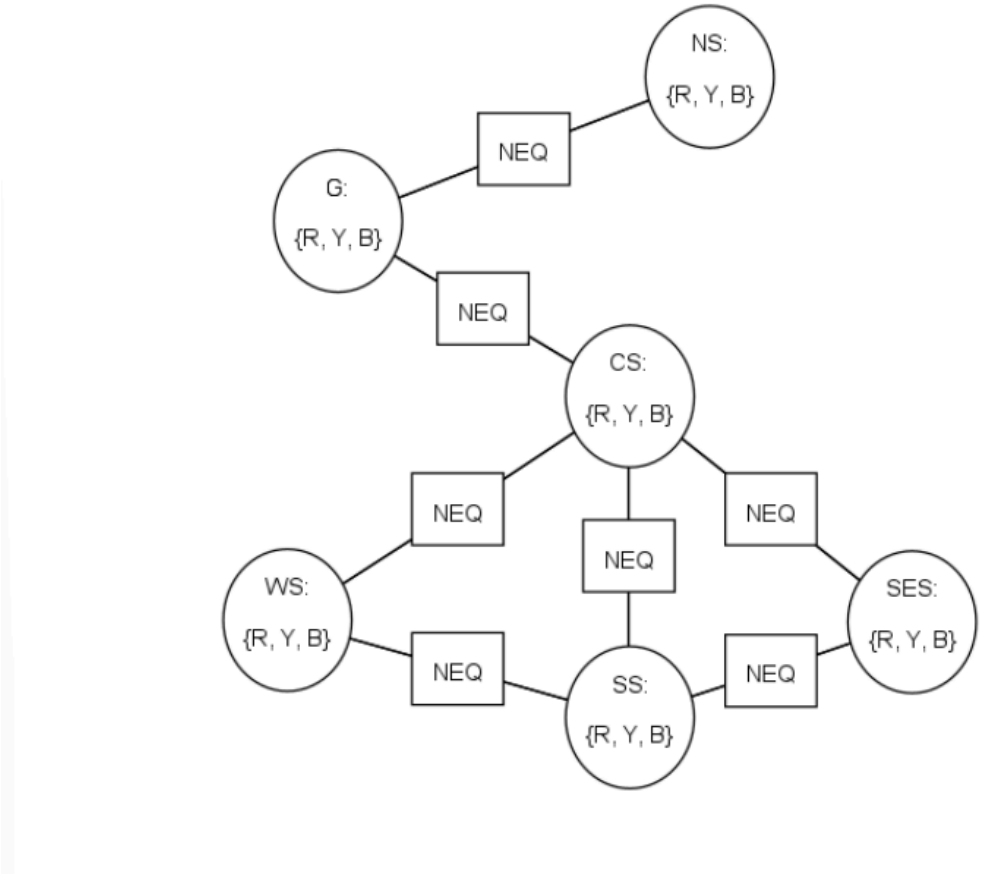
4. **Conditional Logic**: Handling dependencies and "if-then" scenarios in truth tables.

5. **Scalability**: Structuring models that remain efficient even with many variables.

3. Solution Explanation

3.1 Problem 1: Map Coloring (Sulawesi Island)

Objective: Color six provinces such that no adjacent regions have the same color using {Red, Yellow, Blue}.
Variables: G, NS, CS, WS, SS, SES
Domain: {R, Y, B}
Constraints: NEQ constraints for all neighboring provinces, e.g., G ≠ NS, CS ≠ SS, etc.



3.1 My Solution Implementation

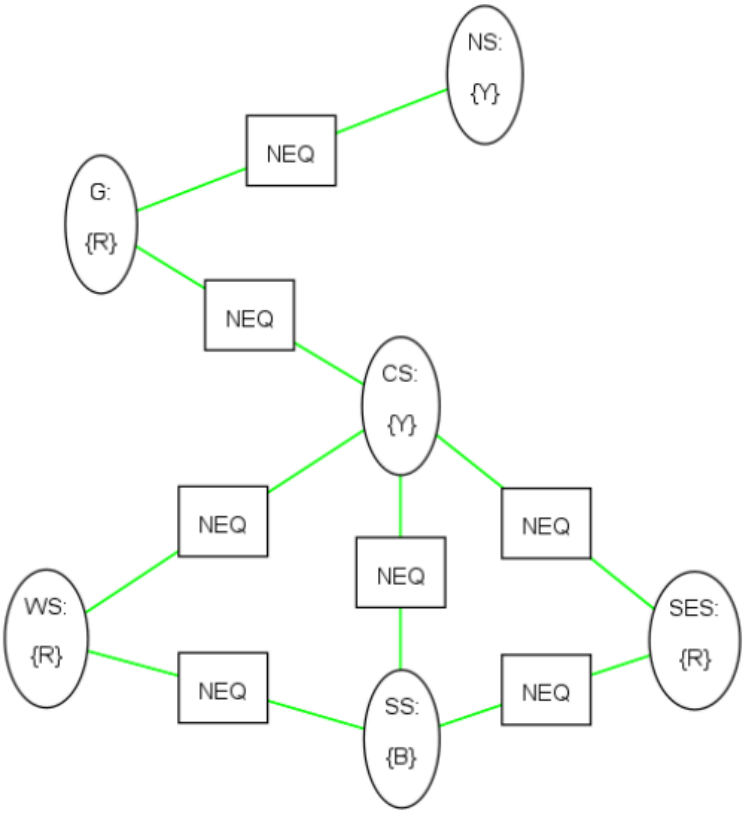
To solve the map coloring problem of Sulawesi Island, I utilized "Not Equal" (NEQ) constraints to ensure that no two adjacent provinces were assigned the same color. I began by analyzing the map to determine which regions shared borders. Based on this, I implemented constraints between all neighboring province pairs.

The key adjacency-based constraints I added were:

- **Gorontalo (G)** is adjacent to **North Sulawesi (NS)** and **Central Sulawesi (CS)** → Constraints: G ≠ NS, G ≠ CS

- **Central Sulawesi (CS)** connects with **West Sulawesi (WS)**, **South Sulawesi (SS)**, and **Southeast Sulawesi (SES)** → Constraints: CS ≠ WS, CS ≠ SS, CS ≠ SES

- **South Sulawesi (SS)** also shares borders with **West Sulawesi (WS)** and **Southeast Sulawesi (SES)** → Constraints: SS ≠ WS, SS ≠ SES



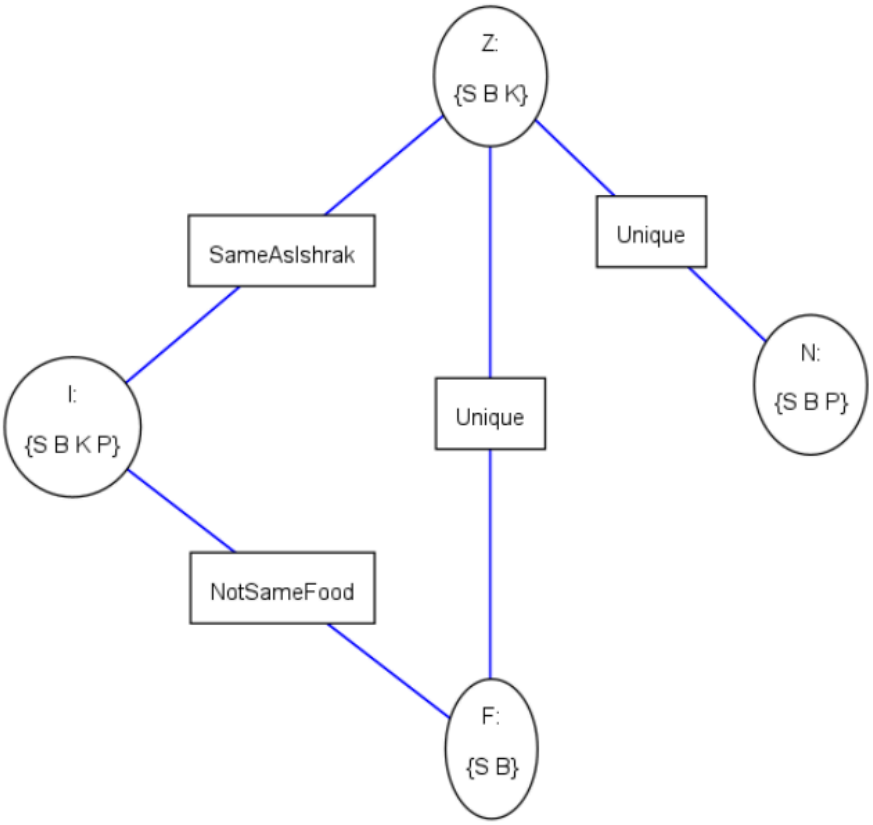
3.2 Problem 2: Restaurant Food Ordering

In this task, I modeled a CSP scenario involving four friends—Zahid, Ishrak, Farabi, and Nafisa—each selecting one dish from a menu of four options: *Special Rice (S)*, *Biriyani Rice (B)*, *Kashmiri Naan (K)*, and *Paratha (P)*. The constraints were based on their individual preferences and logical dependencies between their choices. The main challenge lay in expressing both personal exclusions and interdependencies among their selections in the constraint network.

My CSP Formulation

- **Variables**: Z (Zahid), I (Ishrak), F (Farabi), N (Nafisa)
- **Domains**:
    - Z: {S, B, K} (Zahid doesn’t eat Paratha)

- I: {S, B, K, P} (Ishrak has no restrictions)
- F: {S, B} (Farabi only prefers rice dishes)
- N: {S, B, P} (Nafisa avoids Kashmiri Naan)



3.2 My Solution Implementation

To enforce the rules, I created several custom constraints using truth tables:

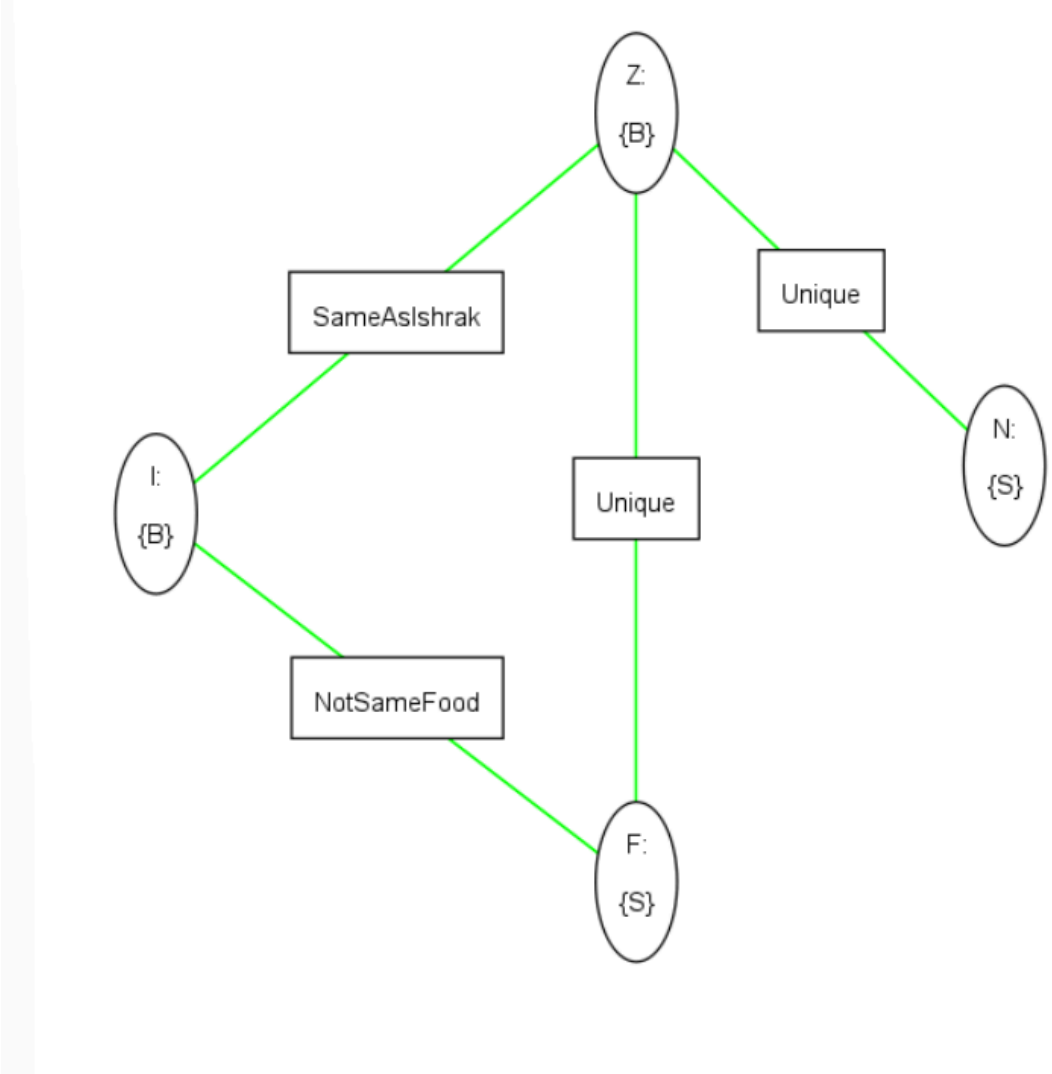
1. **Zahid Copies Ishrak's Choice**: I implemented a constraint to enforce Z = I, ensuring Zahid always selects the same food as Ishrak.

2. **Different Choices for Ishrak and Farabi**: A separate constraint ensured that Ishrak and Farabi must pick different items (I ≠ F).

3. **Overall Uniqueness**: Even though Zahid copies Ishrak, I designed the overall constraint logic so that Zahid's final choice still differs from others in the

group, preventing repetition.

For each constraint, I manually enumerated all valid food combinations to build the corresponding truth tables. This allowed me to precisely model the logical dependencies, especially the chain reactions caused by Zahid's dependency on Ishrak and the exclusion-based preferences of others. Through this, I ensured that the solution remained valid under all specified conditions.



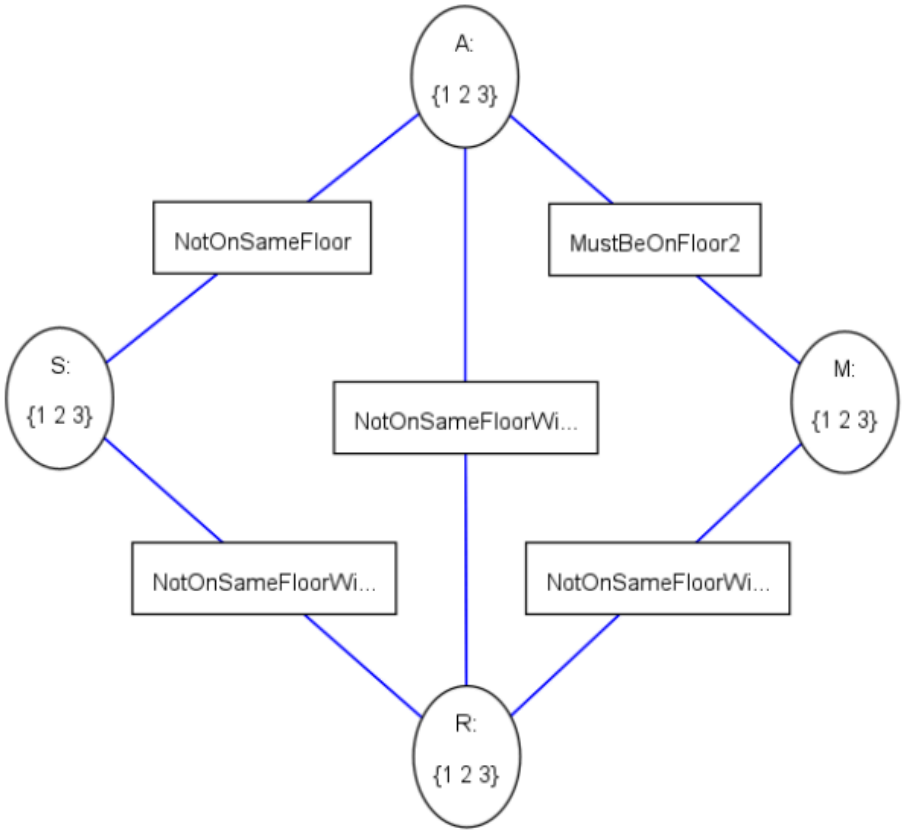
3.3 Problem 3: Apartment Floor Assignment

This task involved assigning four individuals—**Ali**, **Sristy**, **Maliha**, and **Rafid**—to three different floors in an apartment building, subject to several complex conditions.

The main difficulty was in correctly modeling logical dependencies and exclusions, especially the conditional behavior between certain pairs of people and isolation requirements.

My CSP Formulation

- **Variables**: A (Ali), S (Sristy), M (Maliha), R (Rafid)
- **Domain**: {1, 2, 3} for each variable, representing floor levels



3.2 My Solution Implementation

To satisfy the given constraints, I designed a set of custom constraint tables:

1. **Ali and Sristy Must Be on Different Floors**: I created a constraint table ensuring A ≠ S, so they cannot live on the same floor.

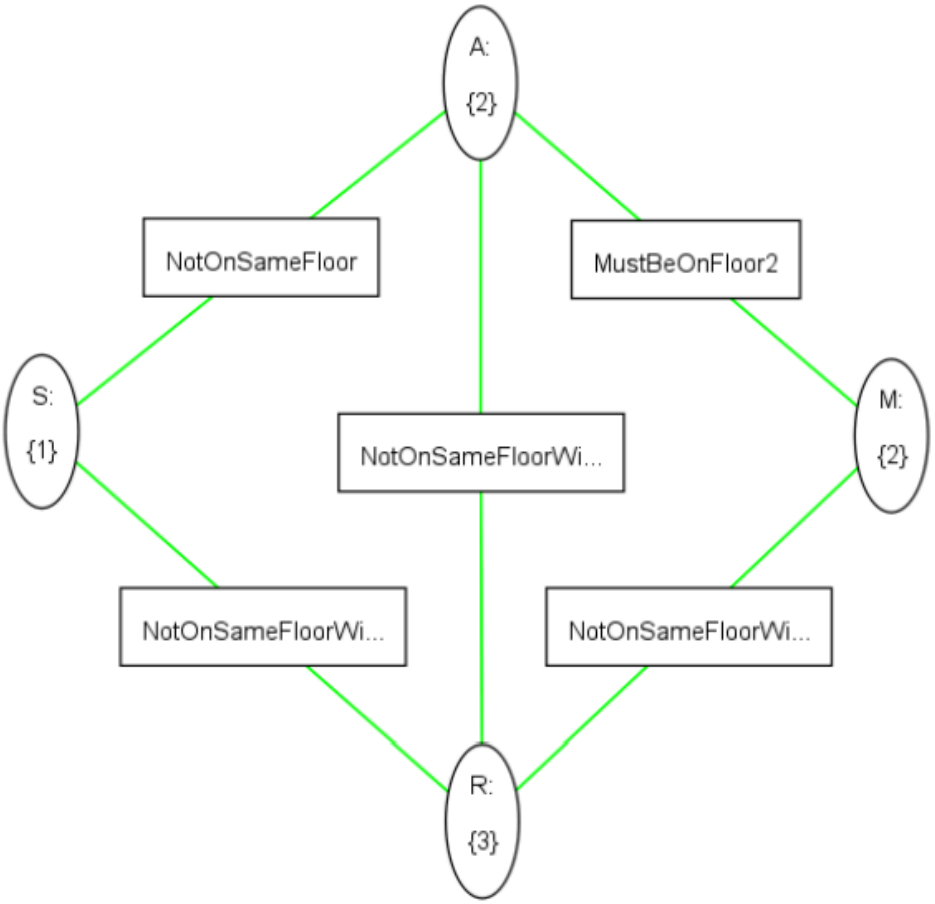
2. **Ali and Maliha's Conditional Rule**: If both Ali and Maliha live on the same floor, that floor must be **Floor 2**. If they are not on the same floor, then at least one of them must be on **Floor 3**. This logic was modeled by constructing all

valid value combinations and including only those that satisfied the rule.

3. **Rafid Must Be Isolated**: I added a constraint that prohibits Rafid from sharing a floor with **any** of the others—meaning R ≠ A, R ≠ S, and R ≠ M simultaneously.

4. **Rafid Must Live Above Maliha**: I encoded the constraint R > M using a LessThan relation in reverse, ensuring Rafid stays on a higher floor than Maliha.

To implement these rules, I manually built truth tables by iterating through all valid floor assignments for each relevant pair. The Ali-Maliha conditional constraint was the most complex, as it required handling "if-then" logic precisely. Careful enumeration and filtering allowed me to enforce each condition accurately without conflicts.

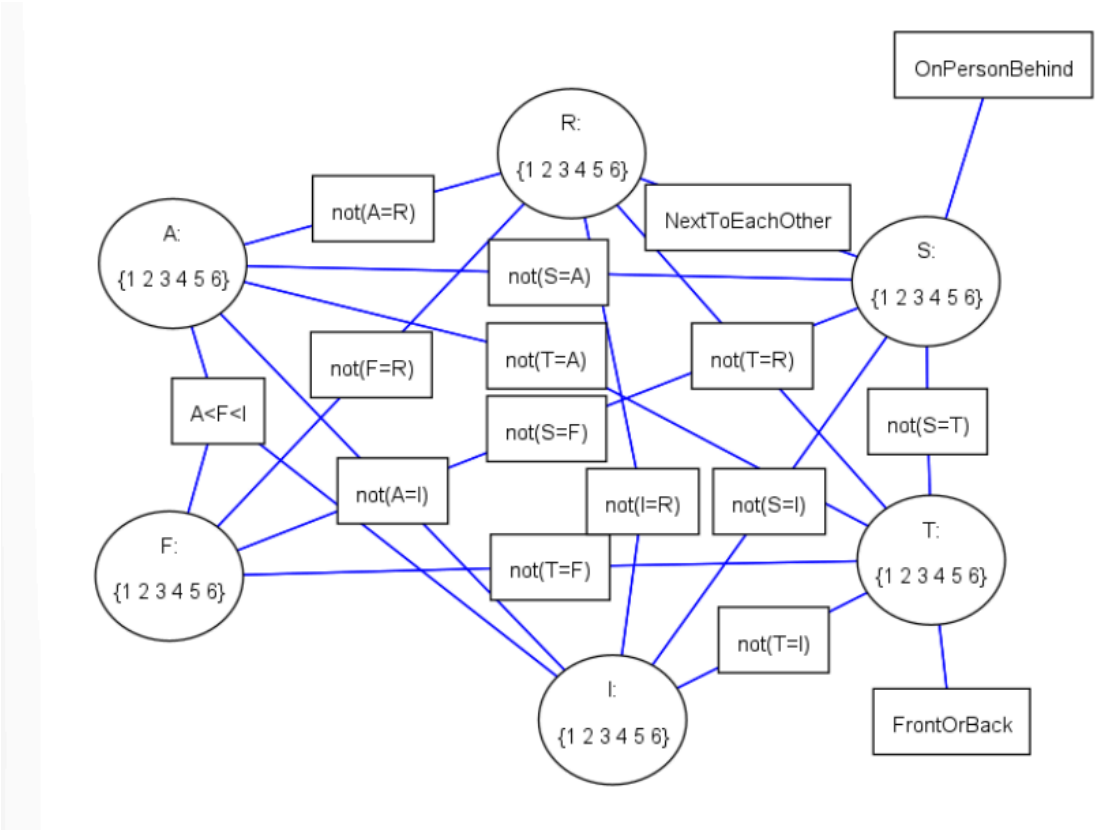


3.4 Problem 4: Queue Positioning

In this task, I was asked to determine the order of six friends—**Rifat**, **Atiq**, **Farhan**, **Ishmam**, **Tabassum**, and **Sabrina**—standing in a line. Each friend must occupy a unique position from 1 to 6. The problem was defined by specific constraints about ordering, adjacency, and fixed positions. What made this task interesting was how the constraints overlapped, requiring careful coordination to avoid conflicts.

My CSP Formulation

- **Variables**: R (Rifat), A (Atiq), F (Farhan), I (Ishmam), T (Tabassum), S (Sabrina)
- **Domain**: {1, 2, 3, 4, 5, 6}, representing queue positions



3.2 My Solution Implementation

To model the positional requirements, I created a mix of built-in and custom constraints:

1. **Ordered Triplet**: I applied a built-in **LessThan** constraint to ensure:
   `A < F < I`

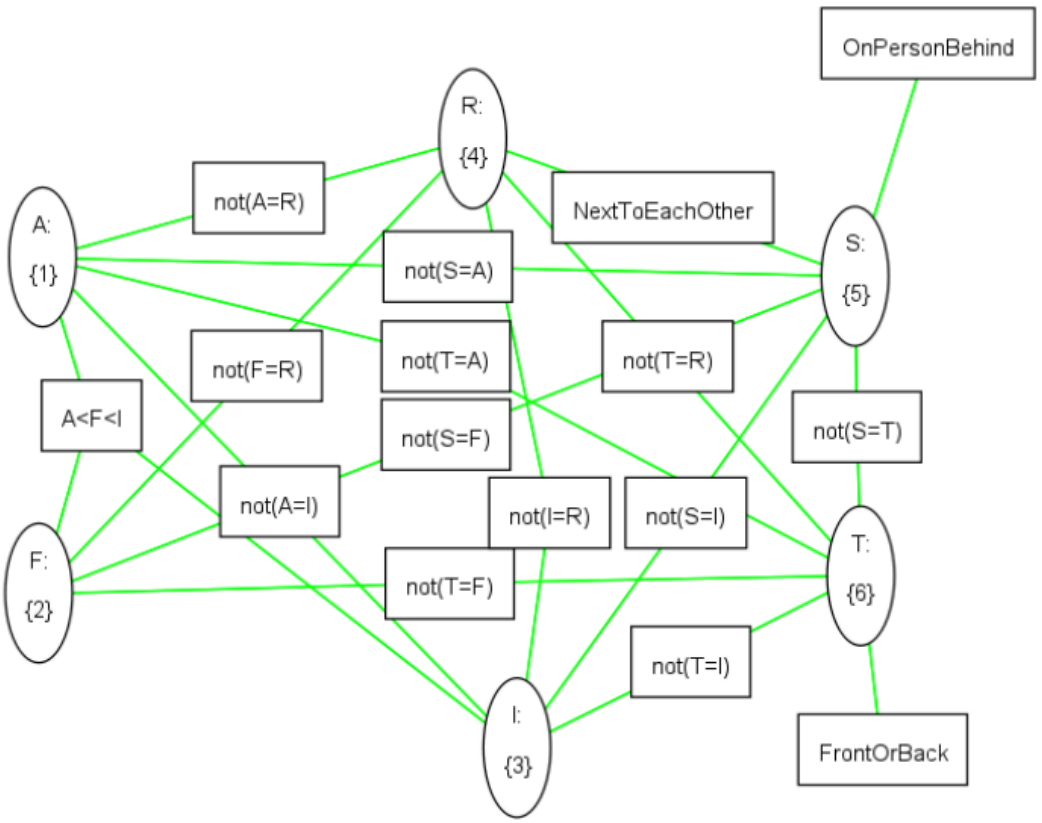
This placed **Farhan** between **Atiq** and **Ishmam** in the lineup.

2. **Rifat and Sabrina Must Stand Together**: I designed a custom truth table to allow only those (R, S) position pairs where the absolute difference is 1 (i.e., adjacent). This involved listing all 36 possible combinations and selecting only the valid adjacent ones.

3. **Tabassum on the Edge**: I constrained **T** to positions 1 or 6 only, using a domain restriction filter to enforce this boundary condition.

4. **Sabrina Fixed at Position 5**: Since only one person can stand behind **Sabrina**, I implemented a constraint fixing her position to exactly 5.

Each of these constraints was modeled carefully to ensure the overall solution space was narrowed without over-constraining. The adjacency condition ($|R - S| = 1$) was particularly important to handle precisely, and the fixed position of Sabrina helped further reduce possible permutations, making the search more efficient.



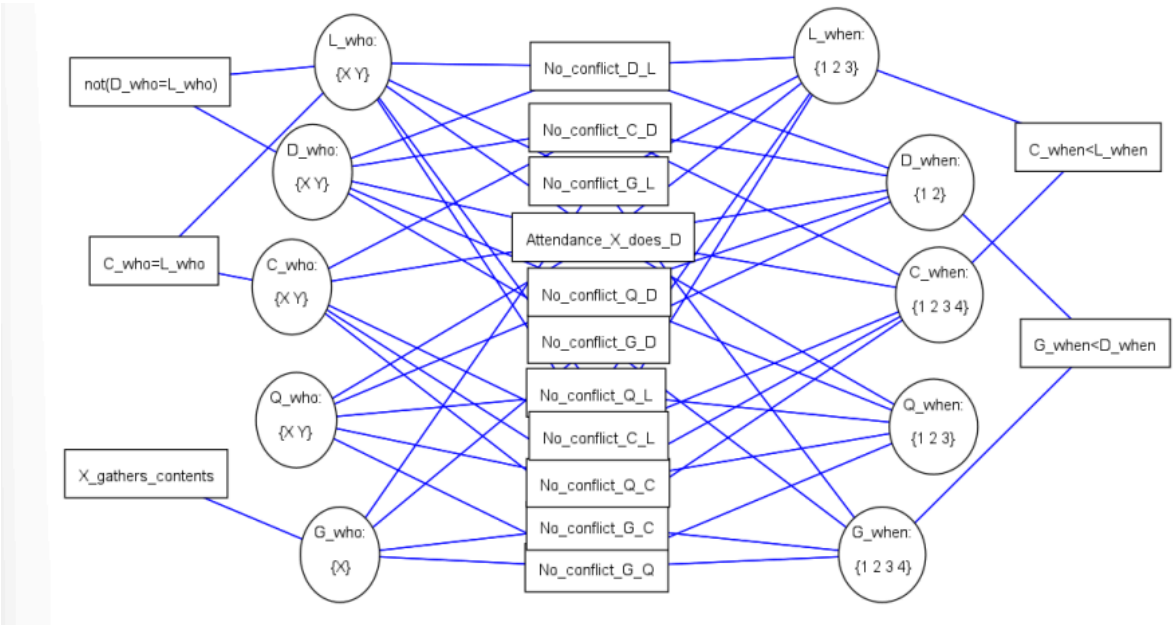
3.5 Problem 5: Faculty Task Scheduling

This was the most intricate task in the lab, involving the scheduling of five tasks—**G**, **Q**, **C**, **D**, and **L**—to two faculty members (**X** and **Y**) over four time slots. Each task came with its own set of requirements regarding who must do it, how long it takes, and in what order it should occur. The main challenge was managing multiple layers of constraints: personnel assignment, timing, precedence, and resource exclusivity.

My CSP Formulation

To effectively model the problem, I separated the variables into two categories:

- **Who Variables**: G_who, Q_who, C_who, D_who, L_who (representing which faculty performs each task)
- **When Variables**: G_when, Q_when, C_when, D_when, L_when (representing when each task is scheduled)
- **Domains**:
    - Who: {X, Y}
    - When: {1, 2, 3, 4}, with restrictions based on task durations

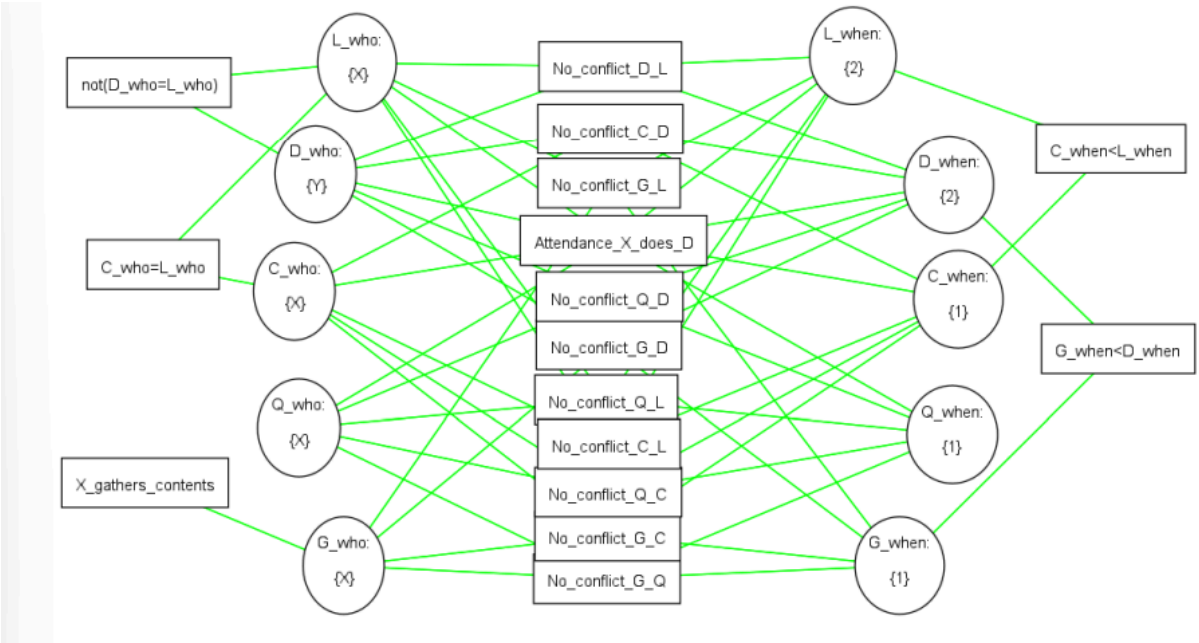


3.2 My Solution Implementation

To capture all the conditions, I developed a comprehensive set of constraints:

1. **Fixed and Complementary Assignments**:
    - Task G is always assigned to **X**.
    - Tasks D and L must be split between X and Y using complementary assignments, ensuring load balance.
2. **Precedence Rules**:
    - C must be scheduled **before** L
    - G must be completed **before** D
      I used LessThan constraints to maintain proper task order.
3. **Time Slot Restrictions**:
    - Task D, which spans two slots, must start in **slot 1 or 2** only.
    - Q and L also have duration-based limits, requiring them to start **by slot 2**.
4. **No Task Overlap (Resource Constraints)**:
    - I constructed custom truth tables between every pair of tasks to prevent the same faculty from doing overlapping tasks in the same time slot.
5. **Lab Attendance Requirement**:
    - When one faculty performs task D (a lab), the other faculty must **remain free** during that time to attend the session.
    - This constraint was the most difficult. I built a large truth table that covered all combinations of assignments and time slots to ensure this rule was never violated.

This problem tested my ability to manage multi-dimensional constraints. By cleanly separating the assignment of faculty (who) from the scheduling (when), I kept the constraint logic modular and more manageable. The attendance condition, involving indirect dependencies, pushed me to be meticulous with truth table verification to ensure the solution was correct under all conditions.



4. Findings and Insights

4.1 Modeling Observations

- Map coloring allowed flexible solutions with minimal constraints.
- Food selection became highly restricted due to interdependencies.
- Floor allocation was constrained by conditional logic.
- Queue ordering required tight positioning rules.
- Scheduling introduced multi-dimensional constraint reasoning.

4.2 Algorithm Behavior

- Arc consistency pruning significantly improved performance across all problems.
- Backtracking search was efficient post-pruning.
- Temporal constraints in scheduling made propagation crucial for feasibility.

5. Challenges Faced and Solutions

5.1 Truth Table Generation

Building correct truth tables for custom constraints (e.g., adjacency, conditional logic) was difficult initially. I solved this through systematic enumeration of value combinations and logic validation.

5.2 Temporal Modeling

Handling overlapping time slots and task durations required separating "who" and "when" variables, linking them via logical constraints.

5.3 Conditional Dependencies

"If-then" constraints in Problems 3 and 5 were initially confusing. I addressed this by encoding logic directly into truth tables, validating each clause explicitly.

6. Additional Insights

6.1 Real-World Applications

These CSP formulations reflect real-world tasks in:

- Resource allocation (e.g., apartment/faculty tasks)
- Scheduling (e.g., staff rosters, class timetables)
- Optimization (e.g., queue management, path planning)

6.2 Scalability Limitations

Constraint tables work well for small domains, but larger problems would require procedural or programmatic constraint encoding for efficiency.

6.3 Alternative Approaches

In Problem 5, I considered binary variables for time-task combinations to simplify logic. This showed that formulation style impacts problem tractability.

7. Conclusion

This lab enriched my understanding of how diverse problems can be encoded as CSPs. From spatial coloring to personnel scheduling, the process of formalizing problems with variables, domains, and constraints proved intellectually engaging and practically valuable. I learned to design and apply arc consistency, backtracking, and custom constraint logic, gaining confidence in modeling real-world scenarios using CSP frameworks.